

College Admission Assistant

1. Introduction

The College Admission Assistant is an AI-powered chatbot designed to help students with queries related to university admissions. It provides information about colleges, admission requirements, eligibility criteria, deadlines, and required documents based on a structured dataset.

The system uses a Retrieval-Augmented Generation (RAG) approach, where relevant information is first retrieved from a dataset using a vector database (FAISS), and then a Large Language Model (LLM) generates accurate and context-aware responses. This reduces incorrect or misleading answers and ensures that responses are grounded in actual data.

The application is built using FastAPI as the backend and HTML, CSS, and JavaScript for the frontend, enabling efficient processing and a user-friendly interface. This project demonstrates how AI can be applied to provide reliable and personalized guidance in the domain of college admissions.

2. Objectives

The main objective of this project is to develop an intelligent chatbot that provides accurate and reliable information related to college admissions using AI techniques.

The specific objectives of the system are:

- To assist students in understanding admission requirements such as GRE, TOEFL, CGPA, and required documents
- To provide information about universities, programs, deadlines, and tuition fees
- To reduce incorrect or generic responses by using a Retrieval-Augmented Generation (RAG) approach
- To enable context-aware and data-driven responses based on a structured dataset
- To support profile-based queries by analyzing user inputs and suggesting suitable universities
- To build a user-friendly interface using HTML, CSS, and JavaScript for easy interaction
- To implement an efficient backend using FastAPI for handling user queries and responses

This project aims to demonstrate the practical application of AI and machine learning techniques in solving real-world problems related to education and decision-making.

3. Scope of the Project

The scope of this project is focused on providing assistance related to college admissions using a predefined dataset. The system is designed to answer queries based on stored information and does not rely on real-time external data sources.

The chatbot supports queries related to:

- University details and available programs
- Admission requirements such as GRE, TOEFL, and CGPA
- Application deadlines and required documents
- Tuition fees and chance of admission
- Profile-based suggestions based on user input

The system is limited to the data available in the dataset and provides responses only within the domain of college admissions. It does not handle unrelated queries and restricts responses to ensure accuracy and relevance.

This project demonstrates how AI can be applied in a specific domain to provide meaningful and context-aware guidance, while maintaining control over the information source.

4. Technologies Used

This project integrates multiple technologies across frontend, backend, and AI/ML components to build an efficient RAG-based chatbot system.

◆ Backend

- **FastAPI:** Used to build the backend API for handling user requests and generating responses efficiently.

◆ Frontend

- **HTML:** Provides the structure of the user interface
- **CSS:** Used for styling and improving the visual appearance
- **JavaScript:** Handles user interaction and communicates with the backend using API calls

◆ AI / Machine Learning

- **Sentence Transformers (all-MiniLM-L6-v2):** Used to convert text data into numerical embeddings
- **FAISS (Facebook AI Similarity Search):** Used as a vector database to store embeddings and perform fast similarity search
- **Groq API (Llama 3.1):** Used as the Large Language Model (LLM) to generate context-aware responses

◆ Programming Language

- **Python:** Used for implementing backend logic, data processing, and integration of AI components

These technologies work together to enable efficient retrieval of relevant information and generation of accurate responses in real time.

5. System Architecture

The system follows a modular architecture that integrates frontend, backend, vector database, and a language model to provide accurate responses.

The overall flow of the system is as follows:

- The user enters a query through the frontend interface
- The query is sent to the FastAPI backend
- The backend converts the query into embeddings using a sentence transformer model
- FAISS vector database retrieves the most relevant documents based on similarity search
- The retrieved context is combined with the user query
- This information is sent to the Groq LLM (Llama 3.1)
- The LLM generates a context-aware response
- The response is sent back to the frontend and displayed to the user

This architecture ensures that responses are generated based on relevant data rather than relying solely on the language model, thereby improving accuracy and reducing incorrect outputs.

6. Dataset Description

The dataset used in this project contains structured information about various universities and their admission details. It serves as the primary knowledge source for the chatbot.

The dataset includes multiple records, where each record represents a university and a specific program offered. The key attributes in the dataset are:

- University name
- Country
- Program offered
- GRE score requirement
- TOEFL score requirement
- Minimum CGPA
- Application deadline
- Required documents
- Tuition fee
- Chance of admission

The dataset is preprocessed and converted into a textual format so that it can be used effectively for embedding generation and retrieval. Each row is transformed into a meaningful sentence describing the university and its admission criteria.

This dataset enables the system to provide accurate and context-based answers by retrieving relevant information during user queries.

7. Data Preprocessing

Data preprocessing is an important step to prepare the dataset for effective retrieval and response generation.

The original dataset is in a structured format (CSV), which is not directly suitable for semantic search. Therefore, each row in the dataset is converted into a meaningful textual representation. This helps the embedding model understand the context better.

The preprocessing steps include:

- Handling missing values by replacing them with appropriate placeholders (e.g., "Not specified")
- Converting each row into a structured sentence containing all relevant details such as university name, program, scores, deadlines, and documents
- Cleaning and formatting the text for consistency
- Storing the processed text as documents for further embedding

Example transformation:

- Original row → structured text:

"University of Toronto in Canada offers Computer Science. GRE: 328, TOEFL: 107, CGPA: 8.62, Deadline: Feb 01, 2025..."

This preprocessing ensures that the data is in a format suitable for embedding generation and improves the accuracy of retrieval in the RAG pipeline.

8. Embedding Generation

Embedding generation is the process of converting textual data into numerical vectors so that it can be used for semantic search and similarity comparison.

In this project, the preprocessed text documents are converted into embeddings using the **Sentence Transformers model (all-MiniLM-L6-v2)**. This model captures the semantic meaning of the text rather than just keywords.

The steps involved are:

- Each processed document is passed through the embedding model
- The model converts the text into a fixed-length numerical vector
- These vectors represent the meaning and context of the text
- Similar texts will have similar vector representations

These embeddings are then stored in the FAISS vector database, which enables efficient similarity search during query processing.

By using embeddings, the system can understand user queries based on meaning rather than exact word matching, improving the relevance and accuracy of retrieved results.

9. Vector Database (FAISS)

FAISS (Facebook AI Similarity Search) is used in this project as a vector database to store and retrieve embeddings efficiently.

After generating embeddings for all the processed documents, they are stored in the FAISS index. This allows the system to perform fast similarity searches when a user submits a query.

The working of FAISS in this system is as follows:

- All document embeddings are stored in the FAISS index
- When a user query is received, it is converted into an embedding
- FAISS compares the query embedding with stored embeddings
- It retrieves the top k most similar documents based on vector similarity
- These retrieved documents are used as context for generating the final response

FAISS enables efficient handling of large datasets and ensures quick retrieval of relevant information, which is essential for real-time chatbot performance.

10. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is the core technique used in this project to generate accurate and context-aware responses.

Instead of relying only on a language model, RAG combines **retrieval** and **generation** to improve the quality of answers.

The process works as follows:

- When a user submits a query, it is converted into an embedding
- The FAISS vector database retrieves the most relevant documents based on similarity
- These retrieved documents form the **context**
- The context, along with the user query, is sent to the Large Language Model (LLM)
- The LLM generates a response based on this context

This approach ensures that the responses are grounded in actual data from the dataset, rather than being purely generated by the model.

The use of RAG helps in:

- Reducing hallucination (incorrect or fabricated answers)
- Improving accuracy and reliability
- Providing domain-specific responses
- Enhancing the overall performance of the chatbot

By combining retrieval and generation, the system delivers meaningful and data-driven answers for college admission queries.

11. Backend Implementation (FastAPI)

The backend of the system is implemented using **FastAPI**, which is responsible for handling user requests, processing queries, and generating responses.

FastAPI is chosen for its high performance, simplicity, and ability to handle API requests efficiently.

The backend performs the following functions:

- Receives user queries from the frontend through API endpoints
- Processes the query and checks whether it is relevant to the domain
- Converts the query into embeddings using the embedding model
- Retrieves relevant documents from the FAISS vector database
- Constructs a prompt using the retrieved context and user query
- Sends the prompt to the Groq LLM (Llama 3.1)
- Returns the generated response back to the frontend

The main API endpoints include:

- `/` → Serves the frontend interface
- `/chat` → Accepts user queries and returns chatbot responses

FastAPI ensures smooth communication between the frontend and the AI components, enabling real-time interaction and efficient processing of user requests.

12. Frontend Implementation

The frontend of the system is developed using **HTML, CSS, and JavaScript**, providing a simple and interactive interface for users to communicate with the chatbot.

The frontend is responsible for:

- Capturing user input through a text input field
- Sending user queries to the backend using HTTP requests (Fetch API)
- Displaying chatbot responses in a structured chat interface
- Maintaining a smooth and user-friendly interaction

Components:

- **HTML:** Defines the structure of the chatbot interface, including input box, send button, and chat display area
- **CSS:** Enhances the visual appearance with styling such as chat bubbles, layout, and responsiveness

- **JavaScript:** Handles dynamic behavior, including sending queries to the FastAPI backend and updating the UI with responses

The frontend communicates with the backend through the `/chat` API endpoint, where user messages are sent and responses are received in real time.

This separation of frontend and backend ensures flexibility, scalability, and ease of maintenance in the overall system design.

13. Query Processing Logic

The query processing logic is responsible for understanding the user's input and determining how it should be handled by the system.

When a user submits a query, the backend processes it through the following steps:

- **Query Validation:** The system first checks whether the query is related to college admissions. If the query is out of domain, an appropriate message is returned.
- **Query Classification:** The query is analyzed to determine its type, such as:
 - General information query (e.g., university details)
 - Listing query (e.g., list of universities)
 - Profile-based query (e.g., GRE, TOEFL-based suggestions)
- **Embedding Conversion:** The validated query is converted into an embedding using the embedding model
- **Similarity Search:** FAISS retrieves the most relevant documents based on the query embedding
- **Context Selection:** Relevant documents are selected and combined to form context for the LLM
- **Response Generation:** The context and query are passed to the LLM to generate a meaningful response

This logic ensures that the system responds appropriately based on the type of query and retrieves the most relevant information for accurate answer generation.

14. Prompt Engineering

Prompt engineering is the process of designing structured instructions that guide the Large Language Model (LLM) to generate accurate, relevant, and well-formatted responses.

In this project, the prompt plays a crucial role in controlling how the chatbot responds to user queries. The prompt is constructed by combining the **retrieved context from the dataset** and the **user's query**, along with specific instructions.

The prompt is designed with the following objectives:

- Ensure responses are based only on the provided dataset context

- Encourage the model to give concise and structured answers
- Allow reasonable inference when exact information is not available
- Prevent generation of irrelevant or out-of-domain responses
- Maintain clarity and readability in the output

The prompt typically includes:

- Role definition (e.g., “You are a college admission assistant”)
- Context retrieved from FAISS
- User query
- Instructions for response format and constraints

By carefully designing the prompt, the system ensures that the LLM generates meaningful, accurate, and user-friendly responses while minimizing incorrect or misleading outputs.

15. Output Generation

Output generation is the final step in the system where the response is created and presented to the user.

After retrieving relevant context and constructing the prompt, the system sends this information to the Large Language Model (LLM) through the Groq API. The LLM processes the input and generates a response based on the provided context and instructions.

The steps involved in output generation are:

- The constructed prompt (context + user query) is sent to the LLM
- The LLM generates a response using the given information
- The response is received by the backend
- The backend may apply formatting or length control if required
- The final response is sent to the frontend
- The frontend displays the response in the chat interface

The system ensures that the output is:

- Relevant to the user query
- Based on the dataset context
- Clear, concise, and easy to understand

This step completes the interaction cycle, providing users with meaningful and context-aware answers to their queries.

16. Features of the System

The system includes several key features that enhance its functionality and user experience.

- **Semantic Search**
Uses embeddings and FAISS to retrieve relevant information based on meaning rather than exact keywords
- **Context-Aware Responses**
Generates answers based on retrieved dataset context, ensuring accurate and relevant information
- **Profile-Based Recommendations**
Analyzes user inputs such as GRE, TOEFL, and CGPA to suggest suitable universities
- **Out-of-Domain Detection**
Identifies and rejects queries that are not related to college admissions
- **Structured Output**
Provides clear and concise responses, often in a structured or summarized format
- **FastAPI Backend**
Enables efficient handling of API requests and real-time response generation
- **Interactive Frontend**
User-friendly interface built with HTML, CSS, and JavaScript for smooth interaction
- **Scalable Architecture**
Modular design allows easy expansion and integration of new features

These features collectively ensure that the system delivers reliable, efficient, and user-friendly college admission guidance.

17. Advantages

The system offers several advantages that make it effective and reliable for providing college admission guidance.

- **Accurate and Data-Driven Responses**
Answers are generated based on a structured dataset, reducing incorrect or misleading information
- **Reduced Hallucination**
The use of RAG ensures that responses are grounded in retrieved context rather than purely generated content
- **Semantic Understanding**
The system understands user queries based on meaning, not just keywords, improving relevance
- **Fast and Efficient Retrieval**
FAISS enables quick similarity search, allowing real-time response generation

- **User-Friendly Interface**
Simple and interactive frontend makes it easy for users to interact with the chatbot
- **Handles Multiple Query Types**
Supports factual queries, listing queries, and profile-based recommendations
- **Modular and Scalable Design**
The architecture allows easy extension and integration of additional features or datasets
- **Domain-Specific Intelligence**
Focused on college admissions, ensuring more relevant and precise answers

These advantages make the system a practical and efficient solution for assisting students in the college admission process.

18. Limitations

Despite its effectiveness, the system has certain limitations that should be considered.

- **Dependence on Dataset Quality**
The accuracy of responses depends on the completeness and correctness of the dataset
- **No Real-Time Data Updates**
The system does not fetch live data, so information may become outdated over time
- **Duplicate Records Impact Output**
Multiple entries for the same university may lead to aggregated or range-based responses
- **Approximate Reasoning**
When exact data is not available, the system may provide estimated answers based on similar entries
- **Limited Domain Coverage**
The chatbot is restricted to college admission queries and does not handle unrelated topics
- **No Personalization Beyond Input**
Recommendations are based only on the provided query and not on long-term user preferences
- **Frontend Simplicity**
The interface is basic and may lack advanced UI/UX features

These limitations highlight areas where the system can be improved for better performance and user experience.

19. Conclusion

The College Admission Assistant is a RAG-based chatbot that provides accurate and context-aware guidance for university admissions. By combining semantic retrieval using FAISS with response generation through a Large Language Model, the system ensures that answers are grounded in actual dataset information.

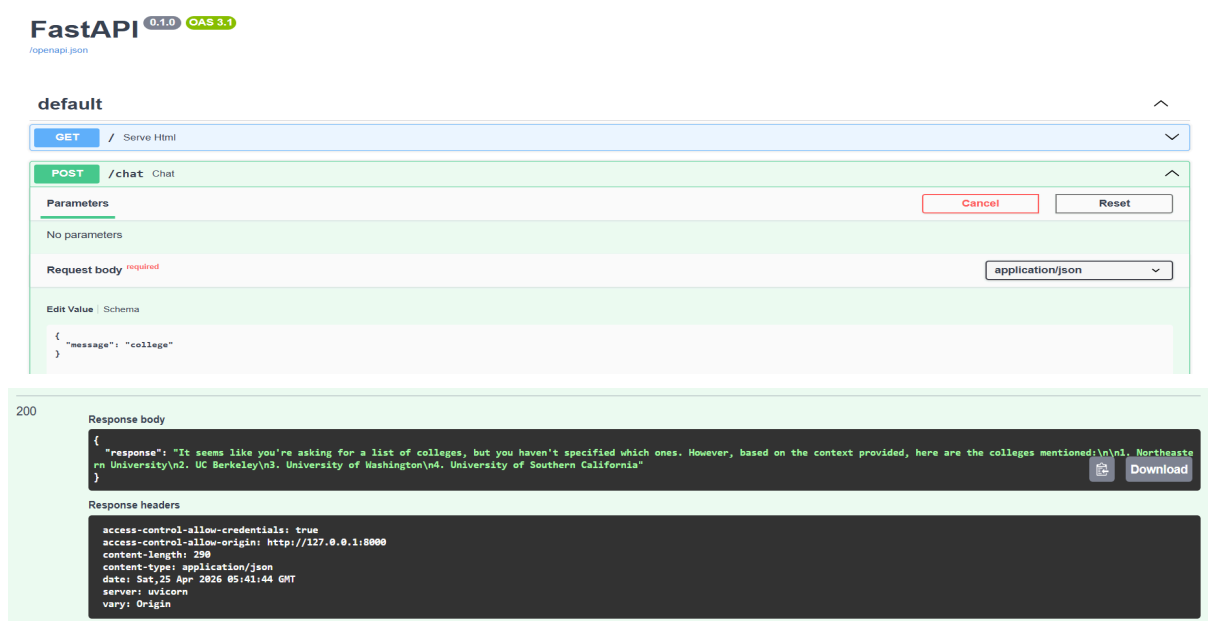
The use of FastAPI for the backend and HTML, CSS, and JavaScript for the frontend enables efficient processing and a smooth user experience. The system successfully handles various types of queries,

including factual, listing, and profile-based questions, while restricting responses to the relevant domain.

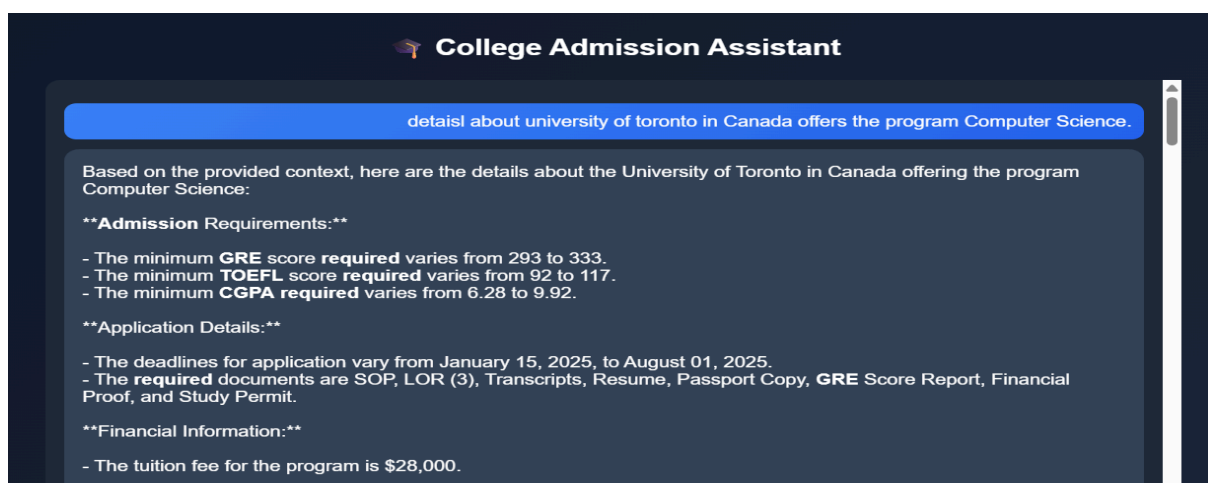
This project demonstrates the practical application of Retrieval-Augmented Generation in building reliable, domain-specific AI systems. It highlights how combining data retrieval with language models can improve accuracy, reduce hallucination, and provide meaningful assistance in real-world scenarios such as college admissions.

20. Screenshots

FastAPI:



Frontend with different responses :



1. What is the GRE requirement for University of Toronto?

Based on the provided context, the **GRE** requirement for University of Toronto varies across different programs. However, it appears that the general trend is that higher **GRE** scores are preferred for more competitive programs.

For the Information Systems program, the minimum **GRE** score is 335. For the Data Science program, the minimum **GRE** score is 331. For the Engineering program, the minimum **GRE** score is 334. For the Computer Science program, the minimum **GRE** score ranges from 298 to 328.

Considering the general trend, it can be inferred that the University of Toronto prefers **GRE** scores above 300 for most of its programs, with the exception of the Computer Science program, which has a lower minimum requirement. However, it's essential to note that the actual **GRE** requirement may vary depending on the specific program and the student's profile.

To provide a reasonable estimate, I would say that the University of Toronto generally prefers **GRE** scores above 320 for most of its programs, with a minimum of 300 for the Computer Science program. However, this is an estimate and may not reflect the actual requirements.

What documents are required for admission to Duke University?

Based on the provided context, the **required** documents for **admission** to Duke University are:

1. Statement of Purpose (SOP)
2. Letters of Recommendation (LOR) - 3
3. Transcripts
4. Resume
5. Passport Copy
6. **GRE** Score Report
7. Financial Proof

These documents are mentioned consistently across all the programs offered by Duke University in the provided context. Therefore, it is reasonable to conclude that these are the **required** documents for **admission** to Duke University.

2. What is the TOEFL score required for MIT?

To determine the **required TOEFL** score for MIT, let's analyze the given data.

We have multiple programs and their corresponding **TOEFL** scores. However, we can see that the lowest **TOEFL** score among the given programs is 92 (Computer Science program with a deadline of Dec 15, 2024) and the highest **TOEFL** score is 115 (AI & ML program).

However, we need to consider the average **TOEFL** score for the Computer Science program, as it has multiple entries. The average **TOEFL** score for the Computer Science program is:

$$(107 + 93 + 112 + 108 + 92 + 93 + 94 + 100 + 112 + 108) / 10 = 101.2$$

The average **TOEFL** score for the Computer Science program is 101.2, which is higher than the lowest **TOEFL** score (92) and lower than the highest **TOEFL** score (115).

Considering the average **TOEFL** score for the Computer Science program, we can estimate the **required TOEFL** score for MIT as follows:

The average **TOEFL** score for the Computer Science program is 101.2, and the average **TOEFL** score for the AI & ML program is 115. We can assume that the **required TOEFL** score for MIT is around the average of these two values.

$$\text{Required TOEFL score for MIT} \approx (101.2 + 115) / 2 \approx 108.1$$

Ask about colleges...



College Admission Assistant

2. List colleges offering Data Science programs

Based on the provided information, the following colleges offer Data Science programs:

1. ****University of Southern California (USA)****: Offers a Data Science program with a **chance of admission** of 0.68 (**GRE** Score: 309, **TOEFL** Score: 92, Minimum **CGPA**: 6.9) and another program with a **chance of admission** of 0.77 (**GRE** Score: 324, **TOEFL** Score: 106, Minimum **CGPA**: 7.42).
2. ****University of California Los Angeles (USA)****: Offers a Data Science program with a **chance of admission** of 0.75 (**GRE** Score: 323, **TOEFL** Score: 113, Minimum **CGPA**: 8.38).
3. ****New York University (USA)****: Offers a Data Science program with a **chance of admission** of 0.79 (**GRE** Score: 321, **TOEFL** Score: 105, Minimum **CGPA**: 7.34) and another program with a **chance of admission** of 0.76 (**GRE** Score: 325, **TOEFL** Score: 96, Minimum **CGPA**: 7.11).
4. ****University of Illinois Urbana-Champaign (USA)****: Offers a Data Science program with a **chance of admission** of 0.68 (**GRE** Score: 320, **TOEFL** Score: 112, Minimum **CGPA**: 7.71).
5. ****Northeastern University (USA)****: Offers a Data Science program with a **chance of admission** of 0.68 (**GRE** Score: 311, **TOEFL** Score: 94, Minimum **CGPA**: 6.6) and another program with a **chance of admission** of 0.73 (**GRE** Score: 323, **TOEFL** Score: 101, Minimum **CGPA**: 6.62).
6. ****Boston University (USA)****: Offers a Data Science program with a **chance of admission** of 0.79 (**GRE** Score: 325, **TOEFL** Score: 101, Minimum **CGPA**: 6.62).

Ask about colleges...

GROQ API USAGE:

API Keys

[+ Create API Key](#)

Manage your project API keys. Remember to keep your API keys safe to prevent unauthorized access.

NAME	SECRET KEY	CREATED	LAST USED	EXPIRES	USAGE (24HRS)
ai_mini	gsk_...kmzQ	24/04/2026	25/04/2026	Never	40 API Calls

